

**Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №6
По дисциплине «Современные платформы программирования»**

Выполнил:
Студент 3 курса
Группы ПО-12
Ишимов А.В.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить приемы тестирования кода на примере использования пакета pytest.

Вариант №7

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

Код:

```
# =====
# Задание 1: Тесты для мини-библиотеки покупок (shopping.py)
# =====

# 3. Фикстура для пустой корзины
@pytest.fixture(name="empty_cart")
def fixture_empty_cart():
    return Cart()

# 1. Добавление товара
def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0

# Тест отрицательной цены
def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Banana", -5)

# Тест подсчёта общей стоимости
def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    assert empty_cart.total() == 15.0

# 2. Тест apply_discount с параметризацией
@pytest.mark.parametrize(
    "discount, expected",
    [
        (0, 20.0),
        (50, 10.0),
        (100, 0.0),
    ],
)
def test_apply_discount(empty_cart, discount, expected):
    empty_cart.add_item("Apple", 20.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected
```

```

@pytest.mark.parametrize("discount", [-10, 150])
def test_apply_discount_invalid(empty_cart, discount):
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)

# 4. Мокирование requests.post в log_purchase
def test_log_purchase():
    with patch("shopping.requests.post") as mock_post:
        item = {"name": "Apple", "price": 10.0}
        log_purchase(item)
        mock_post.assert_called_once_with("https://example.com/log",
            json=item)

# 5. Тесты apply_coupon
def test_apply_coupon_save10(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 90.0

def test_apply_coupon_half(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    apply_coupon(empty_cart, "HALF")
    assert empty_cart.total() == 50.0

def test_apply_coupon_invalid(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    with pytest.raises(ValueError):
        apply_coupon(empty_cart, "INVALID")

def test_apply_coupon_monkeypatch(empty_cart, monkeypatch):
    empty_cart.add_item("Apple", 100.0)
    fake_coupons = {"FAKE50": 50}

    monkeypatch.setattr(shopping, "coupons", fake_coupons)

    apply_coupon(empty_cart, "FAKE50")
    assert empty_cart.total() == 50.0

```

Задание 2: Напишите тесты к реализованным функциям из лабораторной работы №1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Код:

```

# =====
# Задание 2: Тесты к функции plus_one (lab_1.py)
# =====

```

```

# Тривиальный случай — последняя цифра < 9
def test_plus_one_simple():
    assert plus_one([1, 2, 3]) == [1, 2, 4]

# Последняя цифра равна 9 — перенос разряда
def test_plus_one_carry():
    assert plus_one([1, 2, 9]) == [1, 3, 0]

# Несколько девяток подряд — цепочка переносов
def test_plus_one_multiple_carry():
    assert plus_one([1, 9, 9]) == [2, 0, 0]

# Все цифры — девятки: результат должен расширить список
def test_plus_one_all_nines():
    assert plus_one([9, 9, 9]) == [1, 0, 0, 0]

# Одна цифра 0
def test_plus_one_zero():
    assert plus_one([0]) == [1]

# Одна цифра 9
def test_plus_one_single_nine():
    assert plus_one([9]) == [1, 0]

# Число из одной цифры, не 9
def test_plus_one_single_digit():
    assert plus_one([5]) == [6]

# Длинное число без переноса
def test_plus_one_long_number():
    assert plus_one([1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]) == [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

```

Задание 3: Напишите метод `String substringBetween(String str, String open, String close)` выделяющий подстроку относительно открывающей и закрывающей строки.

Спецификация метода:

`substringBetween (None , None, None) = TypeError`

`substringBetween (None, * , *) = None`

`substringBetween (* , None, *) = None`

`substringBetween (* , * , None) = None`

`substringBetween ("", "", "") = ""`

```
substringBetween("", "", "]") = None
substringBetween("", "[", "]") = None
substringBetween(" yabcz ", "", "") = ""
substringBetween(" yabcz ", "y", "z") = " abc"
substringBetween(" yabczyabcz ", "y", "z") = " abc "
substringBetween("wx[b]yz", "[", "]") = "b"
```

Код:

```
# =====
# Задание 3: Тесты к функции substring_between (task_3.py)
# =====

# Все три аргумента None — TypeError
def test_substring_between_all_none():
    with pytest.raises(TypeError):
        substring_between(None, None, None)

# str=None, остальные любые — None
def test_substring_between_str_none():
    assert substring_between(None, "y", "z") is None

# open=None — None
def test_substring_between_open_none():
    assert substring_between("yabcz", None, "z") is None

# close=None — None
def test_substring_between_close_none():
    assert substring_between("yabcz", "y", None) is None

# Все пустые строки — пустая строка
def test_substring_between_all_empty():
    assert substring_between("", "", "") == ""

# str пустая, open пустой, close не пустой — None
def test_substring_between_empty_str_nonempty_close():
    assert substring_between("", "", "]") is None

# str пустая, оба разделителя не пустые — None
def test_substring_between_empty_str_brackets():
    assert substring_between("", "[", "]") is None

# Оба разделителя пустые, str не пустая — пустая строка
def test_substring_between_empty_delimiters():
    assert substring_between("yabcz", "", "") == ""
```

```

# Обычный случай – первое вхождение
def test_substring_between_basic():
    assert substring_between("yabcz", "y", "z") == "abc"

# Несколько вхождений – возвращается первое
def test_substring_between_multiple_occurrences():
    assert substring_between("yabczyabcz", "y", "z") == "abc"

# Квадратные скобки
def test_substring_between_brackets():
    assert substring_between("wx[b]yz", "[", "]") == "b"

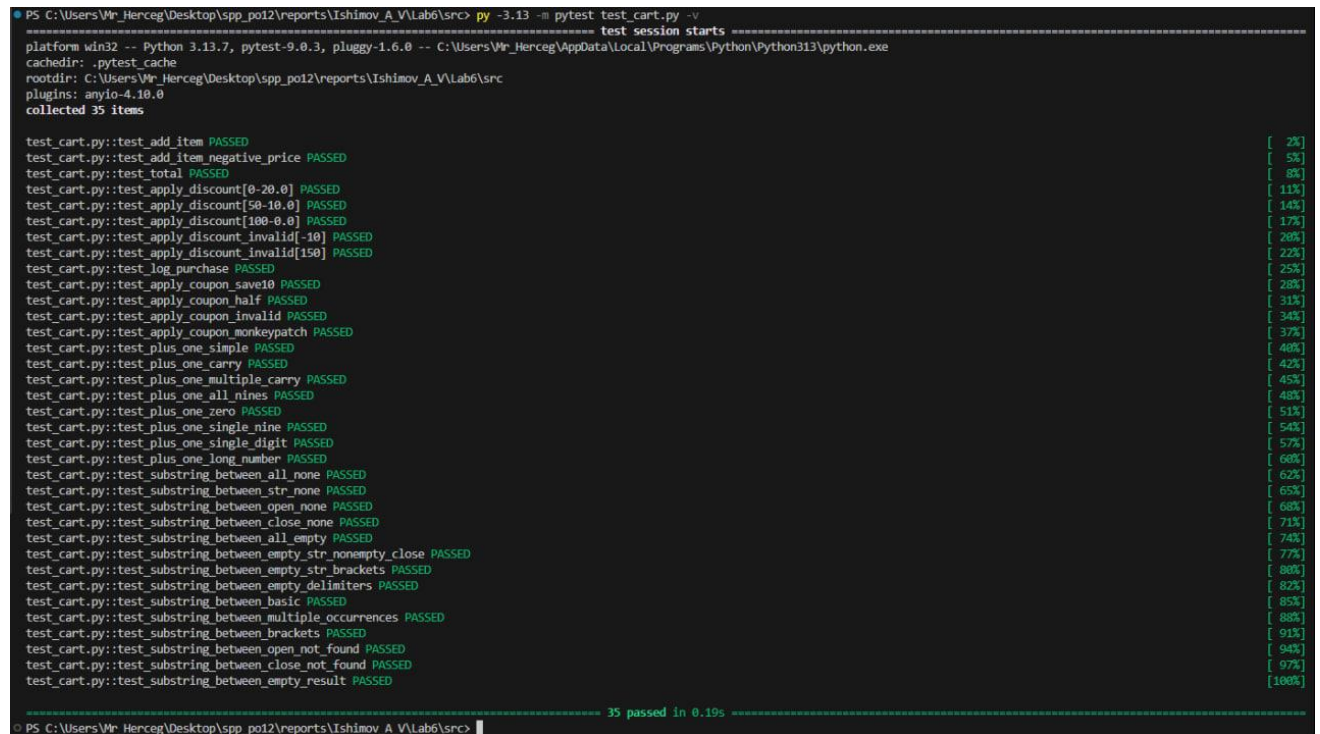
# Открывающая строка не найдена – None
def test_substring_between_open_not_found():
    assert substring_between("hello world", "[", "]") is None

# Закрывающая строка не найдена – None
def test_substring_between_close_not_found():
    assert substring_between("hello[world", "[", "]") is None

# Подстрока между разделителями пустая
def test_substring_between_empty_result():
    assert substring_between("[]", "[", "]") == ""

```

Результат:



```

PS C:\Users\Wn_Herceg\Desktop\spp_po12\reports\Ishimov_A_V\Lab6\src> py -3.13 -m pytest test_cart.py -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Wn_Herceg\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Wn_Herceg\Desktop\spp_po12\reports\Ishimov_A_V\Lab6\src
plugins: anyio-4.10.0
collected 35 items

test_cart.py::test_add_item PASSED [ 2%]
test_cart.py::test_add_item_negative_price PASSED [ 5%]
test_cart.py::test_total PASSED [ 8%]
test_cart.py::test_apply_discount[0-20.0] PASSED [ 11%]
test_cart.py::test_apply_discount[50-10.0] PASSED [ 14%]
test_cart.py::test_apply_discount[100-0.0] PASSED [ 17%]
test_cart.py::test_apply_discount_invalid[-10] PASSED [ 20%]
test_cart.py::test_apply_discount_invalid[150] PASSED [ 22%]
test_cart.py::test_log_purchase PASSED [ 25%]
test_cart.py::test_apply_coupon_save10 PASSED [ 28%]
test_cart.py::test_apply_coupon_half PASSED [ 31%]
test_cart.py::test_apply_coupon_invalid PASSED [ 34%]
test_cart.py::test_apply_coupon_monkeypatch PASSED [ 37%]
test_cart.py::test_plus_one_simple PASSED [ 40%]
test_cart.py::test_plus_one_carry PASSED [ 42%]
test_cart.py::test_plus_one_multiple_carry PASSED [ 45%]
test_cart.py::test_plus_one_all_nines PASSED [ 48%]
test_cart.py::test_plus_one_zero PASSED [ 51%]
test_cart.py::test_plus_one_single_nine PASSED [ 54%]
test_cart.py::test_plus_one_single_digit PASSED [ 57%]
test_cart.py::test_plus_one_long_number PASSED [ 60%]
test_cart.py::test_substring_between_all_none PASSED [ 62%]
test_cart.py::test_substring_between_str_none PASSED [ 65%]
test_cart.py::test_substring_between_open_none PASSED [ 68%]
test_cart.py::test_substring_between_close_none PASSED [ 71%]
test_cart.py::test_substring_between_all_empty PASSED [ 74%]
test_cart.py::test_substring_between_empty_str_nonempty_close PASSED [ 77%]
test_cart.py::test_substring_between_empty_str_brackets PASSED [ 80%]
test_cart.py::test_substring_between_empty_delimiters PASSED [ 82%]
test_cart.py::test_substring_between_basic PASSED [ 85%]
test_cart.py::test_substring_between_multiple_occurrences PASSED [ 88%]
test_cart.py::test_substring_between_brackets PASSED [ 91%]
test_cart.py::test_substring_between_open_not_found PASSED [ 94%]
test_cart.py::test_substring_between_close_not_found PASSED [ 97%]
test_cart.py::test_substring_between_empty_result PASSED [100%]

===== 35 passed in 0.19s =====
PS C:\Users\Wn_Herceg\Desktop\spp_po12\reports\Ishimov_A_V\Lab6\src>

```

Вывод: освоил приемы тестирования кода на примере использования пакета pytest.